Express vs NestJS

Comparing two implementations of the same Product Catalog API, and why NestJS is the recommended foundation

	product-catalog-api	product-catalog-nest-api
Framework	Express 5 + TypeScript 5	NestJS 12 (on Express) + TypeScript 6
Origin	2020 Node/Express/TypeScript setup, modernized in 2026	Rebuilt from the Express version on the official NestJS starter
Database	MongoDB via Mongoose 9	MongoDB via Mongoose 9 (@nestjs/mongoose)
API surface	17 endpoints: categories + products CRUD, search, info, health	The same 17 endpoints, same paths and behaviour

Executive summary

Both projects expose the same 17 endpoints over the same MongoDB data, with the same business rules: unique category names, renames that update products, and no deletion of categories that still have products. Both also run with zero setup by falling back to an in-memory MongoDB. Because the feature set is held constant, the differences below come from the framework itself.

- **NestJS gives structure that Express leaves to convention.** Modules, dependency injection, DTO validation, generated Swagger docs, lifecycle hooks and a test harness are built in. In the Express version, each of these was written by hand or is missing.
- **Measured costs are real but modest.** On this machine, NestJS starts in ~1.5–1.9 s versus ~0.6 s, uses ~60 MB more memory at idle, and its production install is about 7x larger. For database-bound endpoints, throughput is within 2–12% of Express.
- **Quality signals favour NestJS.** It ships with 19 automated tests (the Express version has none), rejects unknown fields, and keeps the API documentation generated from the code, so it can never drift.

Recommendation: choose **NestJS** for this API and for services the team will grow, share and maintain. Keep plain Express for very small, single-purpose services or serverless functions where cold-start time and image size matter most.

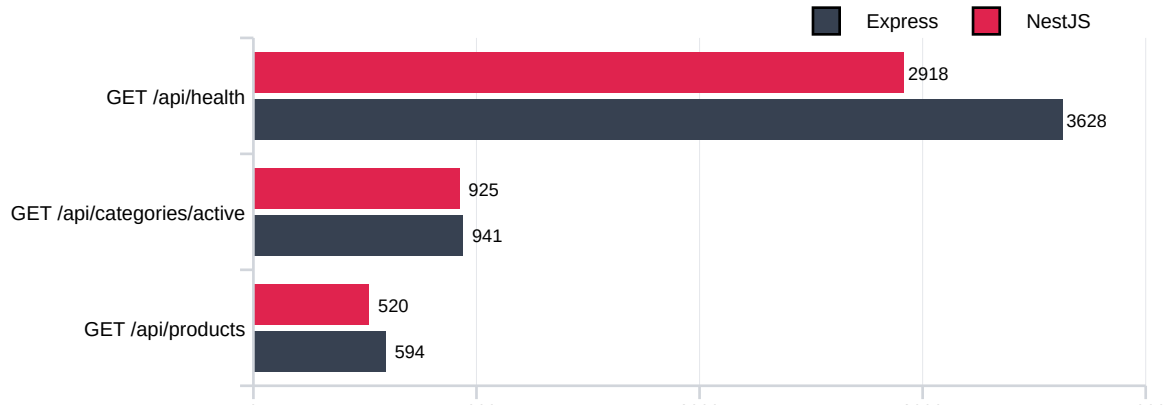
1. The two implementations side by side

Concern	Express version	NestJS version
Project structure	Folders by convention: controller/, data-manager/, models/, helper/	Enforced modules: categories/, products/, seed/, info/, database/, each with controller, service, schema and DTOs
Wiring	Controllers create managers with `new`; models imported directly	Dependency injection: services and models injected through constructors
Input validation	Hand-written: `pick()` allow-list, parseBoolean, toList, Mongoose schema errors	Declarative class-validator DTOs and a global ValidationPipe; unknown fields rejected with 422
API documentation	Hand-maintained tables in README	Swagger UI at /api/docs and OpenAPI JSON, generated from controllers and DTOs
Error handling	Central Express error middleware + custom HttpError class	Global exception filter + built-in exception classes (NotFoundException, ConflictException...)
Startup & shutdown	Manual cleanup registry and signal handlers in main.ts	Lifecycle hooks: onApplicationBootstrap (seeding), onApplicationShutdown (stops in-memory DB)
Route listing	Walks Express internals plus a mount-path registry	Reads decorator metadata with DiscoveryService
Tests	None	2 unit + 17 end-to-end tests (Vitest + Supertest) on a fresh in-memory DB
Dev reload	nodemon + tsx	nest start --watch (TypeScript compiler watch)
Module format	CommonJS	Native ES modules (NestJS 12 standard)

2. Measured comparison

All numbers were measured on the two projects in this repository, on the same Linux machine, Node.js 22.22, against the same MongoDB-compatible database seeded with the same 7 categories and 23 products.

Metric	Express	NestJS	Difference
Source files (excl. tests)	17	26	+9
Source lines (excl. tests)	860	1,308	+52% (DTOs, Swagger annotations, lifecycle and features Express lacks)
Automated tests	0	19 (2 unit + 17 e2e)	e2e suite runs in ~5 s
Runtime dependencies	3	10	+7 (@nestjs/*, class-validator, rxjs...)
Production install size (npm ci --omit=dev)	17 MB	118 MB	~7x (Swagger UI, TypeScript, validator data)
Compiled output (dist/)	168 KB	380 KB	Small either way
Full build time	7.2 s	9.1 s	+1.9 s
Startup to first healthy response	575–597 ms	1,474–1,922 ms	~1 s slower (includes DB connection probe)
Memory (RSS) at idle	87 MB	148 MB	+61 MB
Memory (RSS) after load test	215 MB	320 MB	+105 MB
GET /api/health throughput / p99	3,628 req/s / 27 ms	2,918 req/s / 35 ms	–20% (framework-only path)
GET /api/categories/active	941 req/s / 130 ms	925 req/s / 120 ms	–2% (database-bound)
GET /api/products (23 docs)	594 req/s / 160 ms	520 req/s / 188 ms	–12% (database-bound)



Requests per second (higher is better), 50 concurrent connections, 10 s

How to read this. The health endpoint has no database work, so it isolates framework overhead: NestJS's pipes, filters and DI add roughly 20% per request at this load. On real endpoints, the database dominates and the gap narrows to 2–12%. Both versions served every request without errors. If raw throughput becomes a constraint, NestJS can switch its HTTP layer to Fastify without touching controllers (see section 4).

Caveats. Measured in a cloud container against an in-memory MongoDB-compatible stand-in, because real MongoDB could not be downloaded there, using autocannon with 50 connections for 10 s. Treat the figures as relative, not absolute. Re-run against your production MongoDB before capacity planning.

3. Developer advantages of NestJS

3.1 Architecture is enforced, not remembered

In Express, the controller → manager → model layering exists only because the original author chose it. Nothing stops the next developer from querying MongoDB inside a route handler. NestJS makes the module, controller, service and provider structure part of the framework, and `nest g resource` generates it. A developer who

knows one NestJS codebase can find their way around this one immediately, which shortens onboarding and code review.

3.2 Validation, types and docs from one declaration

The Express version validates input in three places: a hand-written allow-list, ad-hoc query parsers, and Mongoose schema errors translated in middleware. The NestJS version declares the rules once on a DTO. That class is the TypeScript type, the runtime validator and the Swagger schema all at once.

Express: manual allow-list + parsing

```
const CATEGORY_FIELDS = ['category', 'isActive'];

async createCategory(payload: object) {
  const data = pick(payload, CATEGORY_FIELDS)
    as Partial<ICategory>;      // unchecked cast
  await this.assertNameAvailable(data.category);
  return CategoryModel.create(data);
}

function parseBoolean(value, name) {
  if (value === 'true') return true;
  if (value === 'false') return false;
  throw new HttpError(400, `${name} must be...`);
}
```

NestJS: one DTO = type + validation + docs

```
export class CreateCategoryDto {
  @ApiProperty({ example: 'Payments' })
  @MaxLength(100)
  @IsString()
  @IsNotEmpty()
  category: string;

  @ApiPropertyOptional({ default: true })
  @IsOptional()
  @IsBoolean()
  isActive?: boolean;
}

@Post()
create(@Body() dto: CreateCategoryDto) { ... }
```

Result: a request like `{"category":"X","hacker":true}` is rejected with *422 property hacker should not exist* before any code runs. The Express version silently drops unknown fields.

3.3 Dependency injection makes code testable

Express: hard-wired dependencies

```
// controller file
const manager = new CategoriesManager();

// manager file
import { CategoryModel } from '../models/...';
import { ProductModel } from '../models/...';
// To unit-test, you must mock modules
// or run a real database.
```

NestJS: injected, swappable in tests

```
constructor(
  @InjectModel(Category.name)
  private categoryModel: Model<Category>,
) {}

// in a test:
Test.createTestingModule({ imports: [AppModule] })
  .overrideProvider(getModelToken(Category.name))
  .useValue(mockModel)
```

This is why the NestJS project ships with a 17-case end-to-end suite that boots the real application on a throwaway database in about 5 seconds, while the Express version has no tests.

3.4 More advantages for developers

- **Swagger that cannot drift.** /api/docs is generated from the code on every start. Front-end developers can try endpoints in the browser, and the OpenAPI JSON can generate typed client SDKs. In Express, the README tables must be updated by hand.
- **Request pipeline building blocks.** Guards (authentication and roles), interceptors (logging, caching, response mapping), pipes (validation, transformation) and exception filters each have one clear place and order. In Express, all of these are middleware whose order is easy to get wrong.
- **Lifecycle hooks.** Seeding the in-memory database is an `onApplicationBootstrap` hook and stopping it is `onApplicationShutdown`. The Express version needed a hand-rolled cleanup registry and signal handling.
- **Official integrations.** First-party packages for config, JWT/Passport authentication, caching, scheduling, queues, health checks (Terminus), GraphQL and WebSockets all follow the same patterns, so new features don't need new conventions.
- **Consistent errors by default.** Built-in exceptions (NotFoundException, ConflictException...) replace a custom HttpError class.

4. Deployment and operations advantages

Area	What NestJS provides	Why it matters in production
Graceful shutdown	`enableShutdownHooks()` runs every module's shutdown hook on SIGTERM	Kubernetes and container platforms send SIGTERM on every deploy, so connections close cleanly and in-flight work finishes
Health probes	@nestjs/terminus adds readiness and liveness checks for MongoDB, memory and disk	Load balancers stop routing to unhealthy pods. This project already has /api/health reporting database state
Swappable HTTP layer	Runs on Express today; @nestjs/platform-fastify swaps the adapter without changing controllers	Addresses the ~20% framework-overhead gap measured above if throughput ever becomes the bottleneck
Scale-out path	@nestjs/microservices: TCP, Redis, NATS, MQTT, gRPC, RabbitMQ, Kafka transports	Parts of the catalog can move to message-driven services later using the same modules, DI and testing
API contract	OpenAPI document at /api/docs-json, generated at runtime	Can feed API gateways, contract tests in CI and generated clients for front-end and partner teams
Build options	nest build (tsc) by default; SWC or webpack builders available	Faster CI builds, and single-file bundles for smaller container images
Monorepo support	nest g app / nest g library for several apps sharing libraries	Shared schemas, DTOs and auth code across services without copy-paste
Observability	Interceptors hook into every request in one place	Request logging, metrics and tracing (e.g. OpenTelemetry) are added once, not per route

Deployment costs to plan for

- **Image size:** production dependencies are ~118 MB versus ~17 MB. Use a multi-stage Docker build with `npm ci --omit=dev`. Swagger UI and a TypeScript runtime dependency of @nestjs/swagger make up much of the difference.
- **Cold start:** ~1.5–1.9 s versus ~0.6 s. This doesn't matter for long-running containers, but it does for scale-to-zero serverless functions.
- **Memory:** plan for roughly 150 MB idle and 320 MB under load per instance, versus 90/215 MB for Express.

5. Technical reasoning: why select NestJS

Framework choice should follow what the codebase will become, not what it is today. Today it is two resources and 17 endpoints. Products like this usually grow next to need authentication and roles, more resources (orders, pricing, users), background jobs, audit logging and integrations. The question is which foundation keeps that growth cheap and safe.

Criterion	Express	NestJS	Reasoning
Consistency as the team grows	Low	High	Structure is enforced by the framework, not by reviewers
Input safety	Medium	High	Declarative DTO validation on every route, unknown fields rejected
Testability	Low	High	Dependency injection + testing module; e2e suite already in place
API documentation	Manual	Automatic	OpenAPI generated from code, always current
Adding auth, caching, jobs	Choose and integrate libraries	First-party modules	Same patterns for every concern
Raw speed / footprint	Better	Good	~1 s slower start, +60–105 MB memory, 2–20% lower throughput
Learning curve	Lower	Higher	Decorators, DI and modules take time to learn, but the skills transfer between NestJS projects

The trade-off is clear. NestJS costs a little runtime efficiency and a larger install. In return, the most expensive part of a long-lived service gets cheaper: the engineering time spent understanding, changing, testing and documenting it. For a database-backed API, the measured runtime cost is small (2–12% on real endpoints), and the development and operations benefits grow with every feature and every new team member.

When plain Express is still the better choice

- A tiny, single-purpose service (a webhook receiver, a proxy) that will stay a few routes forever.
- Serverless functions where cold-start time and package size dominate cost and latency.
- Extreme throughput on very thin endpoints, unless NestJS on Fastify closes the gap.

Recommended next steps

- Adopt **product-catalog-nest-api** as the main codebase. Clients see the same endpoints, two differences: unknown body fields now return 422, and error bodies gain a `statusCode` field.
- Add @nestjs/config with schema validation for environment variables, and @nestjs/terminus for readiness probes.
- Add authentication with guards (JWT), then roles on write endpoints.
- Run `npm run test:e2e` in CI on every pull request, and publish the OpenAPI document as a build artifact.
- Containerize with a multi-stage Dockerfile, and benchmark against the production MongoDB before sizing instances.